Day 138: Evaluating Generative Models: Real FID/IS-Style Metric Computation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 6 | Complete study guide

Watch: What is Fréchet Inception Distance (FID)? (Data Science Made Easy) | Frechet Inception Distance and Inception Score - AI Bits and Pieces (Shayan Fazeli)

Script: `day138_evaluating_generative_models.py` · every number in this guide is copied by program from `day138_run_output.txt`, the real output of that script.

The standard scores for generated images, built from scratch on a small digit classifier: the Inception Score, the Frechet distance (FID), and k-nearest-neighbour precision and recall. Each was tested on damage done on purpose, so it is known what it can see. The Inception Score rated 200 copies each of ten images above real digits (9.598 against 9.407); FID (205.857) and recall (0.000) caught it. Exact copies of training images scored like real test digits on every feature-based score (FID 3.161 against 2.996); only the pixel distance to the nearest training image (0.00) caught them. A first draft also produced a wrong lesson: MNIST's test set is not stored in random order, and scoring its first n images mixed a sample-size effect with a change of writers.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

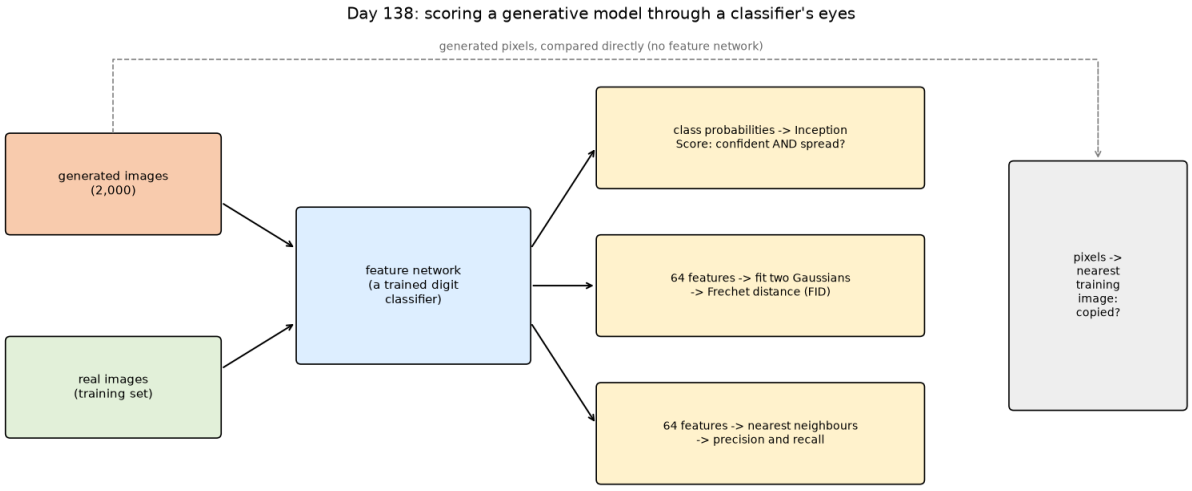
A generative model's output cannot be scored against a right answer: there is no correct image for a noise vector. The standard scores instead look at the **set** of generated images through a trained classifier. The **Inception Score (IS)** asks whether each image is classified confidently and the classes are spread out. The **Frechet distance (FID)** fits a Gaussian to the classifier's features of real images and another to those of generated images, and measures how far apart they are. **Precision** is the share of generated images that land near some real image; **recall** is the share of real images that have some generated image near them.

The originals use Google's Inception network on photographs. Here the classifier is a small network trained on MNIST, so these are 'FID/IS-style' scores: the same formulas, a different feature extractor, and numbers not comparable with published ones.

REAL-WORLD SCENARIO

Judging a forger's paintings without the originals. An expert asks: does each one clearly look like some painter's work (confidence), do they cover all the painters (spread), are they all distinct from each other (variety), and are any of them simply traced from a known painting (copying)? No single question catches every kind of bad forger.

SYSTEM DIAGRAM



Three scores read the feature network; the copying test reads the pixels directly.

Today's part for the week's build

`day138_exercise.py` is the scoring kit (the matrix square root, the Frechet distance, the Inception Score, precision and recall, the feature network and the Scorer), checked by hand: two 1-D Gaussians, hand-made probability tables, and real test digits against noisy ones, with a smaller feature network trained for one epoch.

ACTUAL OUTPUT

```
N(0, 1) against N(3, 4): Frechet distance 10.000000; by hand 3^2 + (1 - 2)^2 = 10
Inception Score: ten confident, evenly spread classes 10.0000; always a confident 3 1.0000; always
    unsure 1.0000
feature network, 1 epoch on 10,000 images: test accuracy 0.8940
  real test digits: IS 6.592, FID 5.492, precision 0.873, recall 0.858, nearest 4.69
   + noise, sd 0.3: IS 3.724, FID 34.842, precision 0.639, recall 0.094, nearest 7.36
```

Data: MNIST (70,000 handwritten digits, cached by Day 126). Day 136's layers, Day 134's AdamGAN, Day 119's distances and Day 116's softmax are imported unchanged; Day 134's dense GAN is Day 136's layer-stack version.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: the formulas where the answer is known

DEFINITION — start here, no prior knowledge assumed

Two of the scores have closed forms worth checking first. For two 1-D Gaussians the Frechet distance is $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$. The Inception Score is between 1 and the number of classes: 1 when every image gets the same verdict or no image gets a confident one, the number of classes when each image is one confident class and the classes are evenly used.

REAL-WORLD SCENARIO

Testing a new bathroom scale with known weights before trusting it with anything else.

MATHEMATICS

$$FID = |\mu_1 - \mu_2|^2 + Tr(\Sigma_1) + Tr(\Sigma_2) - 2\,Tr((\Sigma_1\Sigma_2)^{1/2}), \quad Tr((\Sigma_1\Sigma_2)^{1/2}) = \Sigma\,\sqrt{eig}(\Sigma_1^{1/2}\,\Sigma_2\,\Sigma_1^{1/2})$$

$$IS = exp(\,mean_x\, KL(p(y|x)\,||\,p(y))\,), \quad p(y) = mean_x\, p(y|x)$$

IMPLEMENTATION

```
def sqrtm_psd(S):
    w, V = np.linalg.eigh(S)
    return (V * np.sqrt(np.clip(w, 0, None))) @ V.T
def frechet(mu1, S1, mu2, S2):
    A = sqrtm_psd(S1); M = A @ S2 @ A
    tr_sqrt = np.sqrt(np.clip(np.linalg.eigvalsh((M + M.T) / 2), 0, None)).sum()
    return float(((mu1 - mu2) ** 2).sum() + np.trace(S1) + np.trace(S2) - 2 * tr_sqrt)
def gauss(F):
    F = F.astype(np.float64); return F.mean(0), np.cov(F, rowvar=False)
def inception_score(P):
    P = np.clip(P.astype(np.float64), 1e-12, 1); py = P.mean(0)
    return float(np.exp((P * (np.log(P) - np.log(py))).sum(1).mean()))
```

ACTUAL OUTPUT

```
1-D: N(0, 1) against N(3, 4): formula (mu1 - mu2)^2 + (sd1 - sd2)^2 = 9 + 1 = 10; frechet() gives 10.000000
square root check: |sqrtm(S1)^2 - S1| = 2.1e-14;  Tr (S1 S2)^(1/2) by two routes: 23.29688448 and 23.29688448
200,000 samples of N(0, diag(1, 4, 9)) against the true Gaussian: 0.00028 (0 in the limit)
Inception Score: every image a confident, different class, evenly spread: 10.0000 (10 = the number of classes)
   every image a confident 3:                    1.0000 (1: no spread)
   every image unsure, 0.1 on each class:        1.0000 (1: no confidence)
   confident, but only digits 0-4:               5.0000 (5 = the classes used)
Actual output from this lesson's run.
```

What it means

The 1-D case gives 10.000000, as the formula says. The trace of the matrix square root agrees by two routes (23.29688448 both ways), and a fitted Gaussian from 200,000 samples is 0.00028 from the true one: close to zero, but not zero, which matters in section 3. The four probability tables give exactly 10, 1, 1 and 5. The two different failures behind the two 1s, every image the same digit and every image unsure, get the same score.

WHY CHOOSE THIS?

Closed forms that can be checked, and one number each.

WHY NOT THIS? (TRADEOFFS)

Each number summarises a whole set of images, so different failures can look alike.

2. The feature network

DEFINITION — start here, no prior knowledge assumed

Every score here sees images through a classifier. A small convolutional network is trained on MNIST's labels: two strided convolutions (Day 136's layers), a dense layer of 64 units whose outputs are the features, and a final layer of 10 class scores.

REAL-WORLD SCENARIO

A wine critic is only as useful as their palate: a judge who cannot tell digits apart cannot say whether generated digits look right.

SPECIFICATION

Network	28 × 28 → conv 16 × 14 × 14 → conv 32 × 7 × 7 → 64 features → 10 classes
Training	3 epochs on the 60,000 training images, Adam at 0.001, softmax cross-entropy

IMPLEMENTATION

```
def feature_net(seed=0):
    r = np.random.RandomState(seed)
    body = d136.Net([d136.Act((-1, 1, 28, 28)), d136.Conv(1, 16, 4, 2, 1, r), d136.Act("lrelu"),
        d136.Conv(16, 32, 4, 2, 1, r),
                     d136.Act("lrelu"), d136.Act((-1, 32 * 49)), d136.Dense(32 * 49, 64, r),
                         d136.Act("relu")])
    head = d136.Net([d136.Dense(64, 10, r)])
    return body, head
```

ACTUAL OUTPUT

```
28 x 28 -> conv 4 x 4 stride 2 -> 16 x 14 x 14 -> conv -> 32 x 7 x 7 -> dense -> 64 features -> 10
    classes
trained 3 epochs on the 60,000 training images (58 s): test accuracy 0.9829
Actual output from this lesson's run.
```

What it means

Test accuracy 0.9829. Its 64 features are what FID, precision and recall compare.

WHY CHOOSE THIS?

A judge trained on the same kind of data as the generator, so its features are about digits.

WHY NOT THIS? (TRADEOFFS)

It was trained on the training images, which may make it see them slightly differently from new ones.

3. Real against real: a perfect generator, and which images are scored

DEFINITION — start here, no prior knowledge assumed

A perfect generator would produce new images from the same distribution as the real ones. Real test images stand in for it, scored against references from the training set: the FID Gaussian from training images 0–9,999, and 2,000 of them for precision and recall. Five sizes, from 500 to all 10,000 test images.

REAL-WORLD SCENARIO

Two polls of the same town differ a little just by chance, and more if one of them only asked people on one street.

IMPLEMENTATION

```
perm = 60000 + np.random.RandomState(0).permutation(10000)
rows, sizes = [], (500, 1000, 2000, 5000, 10000)
print(f"{'set scored':>40} {'IS':>7} {'FID':>8} {'precision':>10} {'recall':>7} {'nearest':>8}")
for n in sizes:
    s = sc(X[perm[:n]]); rows.append((n, s)); print(f"{f'{n:,} real test images, random
        order':>40} {s[0]:7.3f} {s[1]:8.3f} {s[2]:10.3f} {s[3]:7.3f} {s[4]:8.2f}")
print("FID falls as the scored set grows, with nothing about the images changing: a fitted
    covariance from few samples is noisy,")
print("and the noise adds to the distance. Scores are only comparable at the same sample size;
    2,000 is used below")
pre = [fid_only(sc, X[60000:60000 + n]) for n in sizes]
```

ACTUAL OUTPUT

```
references: the real Gaussian from training images 0-9,999 (FID); 2,000 of them for precision and recall (k =
    3);
the nearest-image test searches training images 10,000-29,999, in pixels
                              set scored      IS      FID  precision  recall  nearest
     500 real test images, random order   9.319    6.478      0.860   0.907     4.49
   1,000 real test images, random order   9.380    3.760      0.850   0.887     4.49
   2,000 real test images, random order   9.407    2.996      0.855   0.884     4.49
   5,000 real test images, random order   9.419    2.250      0.855   0.884     4.49
  10,000 real test images, random order   9.408    2.060      0.855   0.884     4.49
FID falls as the scored set grows, with nothing about the images changing: a fitted covariance from few
    samples is noisy,
and the noise adds to the distance. Scores are only comparable at the same sample size; 2,000 is used below
the first draft scored the first n test images in MNIST's stored order instead; FID for the same sizes: 17.943
    16.113 14.338 12.006 2.060
the stored order is not random: test images 0-4,999 against 5,000-9,999: FID 12.006 and 6.146 (5,000 in random
    order: 2.250),
named correctly 0.9738 and 0.9920, average ink 0.1221 and 0.1429
Actual output from this lesson's run.
```

What it means

In random order, FID falls as more images are scored: 6.478 at 500, 2.996 at 2,000, 2.060 at 10,000. The images are all real; the fall comes from fitting a 64 × 64 covariance to few samples, whose noise adds to the distance. FID values are only comparable at the same sample size, so 2,000 is used below.

The first draft of this section scored the **first** n test images in the order MNIST stores them. Its FIDs were 17.943, 16.113, 14.338, 12.006 and 2.060, and it printed that the fall was all sample size. It was not: the drop from 12 at 5,000 to 2 at 10,000 was too steep, so it was checked. The stored order is not random. The first and second halves of the test set score FID 12.006 and 6.146 on their own, against 2.250 for a random 5,000; the feature network names them correctly 0.9738 and 0.9920 of the time; their average ink is 0.1221 and 0.1429. The two halves differ measurably, for reasons not examined here, and FID saw it. Any sample taken from a stored dataset has to be random.

WHY CHOOSE THIS?

FID is sensitive: it noticed a difference between two halves of real test data.

WHY NOT THIS? (TRADEOFFS)

It is biased upwards for small samples, and that bias depends on the sample size, not on the images.

4. Damage done on purpose: which score sees which failure?

DEFINITION — start here, no prior knowledge assumed

Real test images are damaged in known ways and scored: Gaussian noise, a blur, half the digits removed, every digit reduced to a single image repeated 200 times (collapse within a class), and exact copies of training images (memorisation).

REAL-WORLD SCENARIO

Testing a smoke alarm with smoke, steam and burnt toast, to learn which ones it actually reacts to.

SPECIFICATION

Base	2,000 test images in random order (section 3)
Damage	noise sd 0.1 and 0.3 (clipped to [0, 1]); 3 × 3 box blur; digits 0–4 only; 10 images × 200; training images 10,000–11,999
Copying test	pixel distance to the nearest of training images 10,000–29,999

IMPLEMENTATION

```
rng = np.random.RandomState(0); T, yt = X[perm], y[perm]; base = T[:2000]
low = T[yt < 5][:2000]
one = np.concatenate([T[yt == d][:1].repeat(200, 0) for d in range(10)])
sets = [("real test images", base), ("+ noise, sd 0.1", np.clip(base + 0.1 *
    rng.randn(*base.shape), 0, 1).astype(np.float32)),
        ("+ noise, sd 0.3", np.clip(base + 0.3 * rng.randn(*base.shape), 0,
            1).astype(np.float32)), ("3 x 3 blur", blur(base).astype(np.float32)),
        ("digits 0-4 only", low), ("one image per digit, x 200", one), ("copies of training
            images", X[10000:12000])]
```

ACTUAL OUTPUT

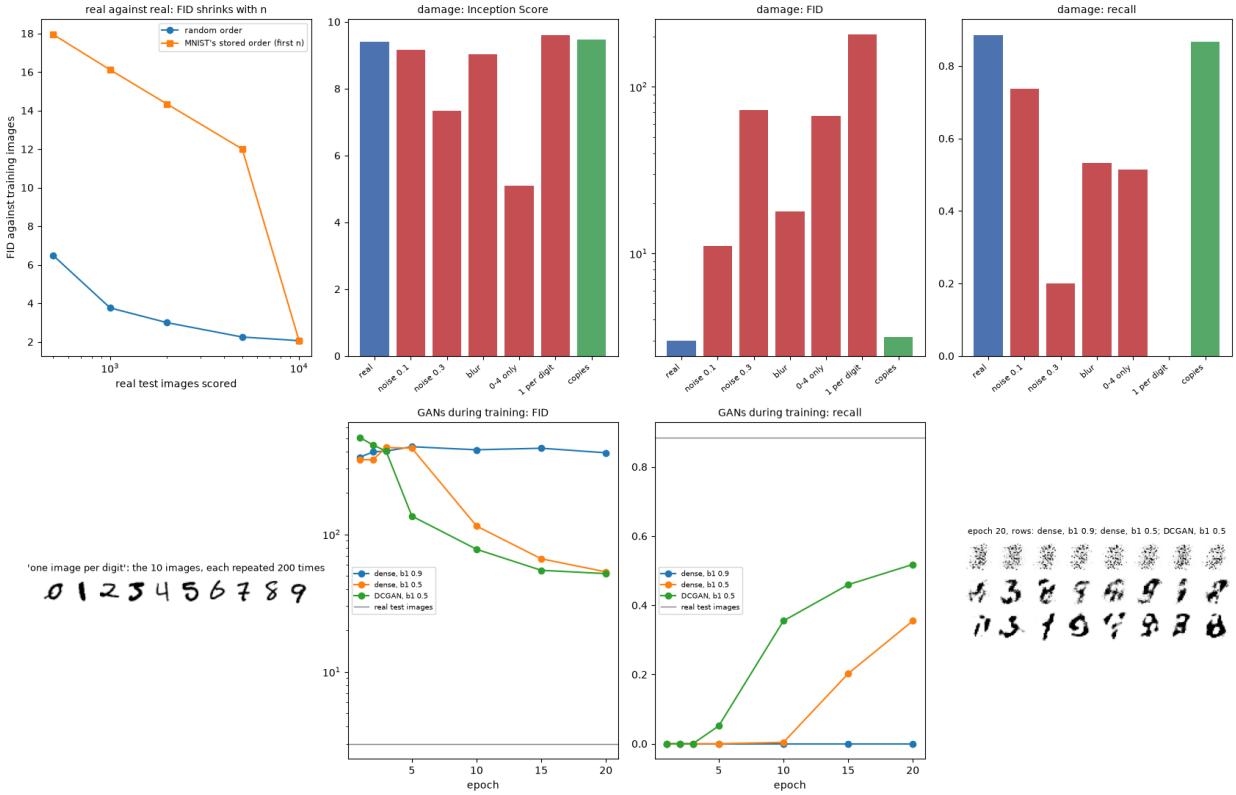
```
2,000 images in each set, from the test set in random order; the nearest-image test looks in training images
    10,000-29,999,
which the last set was copied from
                    set scored      IS      FID  precision  recall  nearest
               real test images   9.407    2.996      0.855   0.884     4.49
                + noise, sd 0.1   9.168   11.097      0.903   0.736     4.85
                + noise, sd 0.3   7.337   72.549      0.872   0.201     7.24
                     3 x 3 blur   9.038   17.932      0.933   0.532     4.19
                 digits 0-4 only   5.086   66.756      0.855   0.513     4.35
    one image per digit, x 200   9.598  205.857      0.700   0.000     4.46
     copies of training images   9.479    3.161      0.858   0.867     0.00
Actual output from this lesson's run.
```

What it means

FID rose with every kind of damage: 11.097 and 72.549 with noise, 17.932 blurred, 66.756 with half the digits gone, 205.857 for ten images repeated. The Inception Score halved when half the digits were gone (5.086), but rated the ten repeated images higher than real digits (9.598): each is a confident, different class, which is all IS checks. Recall caught the missing variety (0.000).

Precision did not behave like a quality score. It went **up** with noise of sd 0.1 (0.903) and with the blur (0.933), from the real images' 0.855. A likely reason, not tested here: damaged images fall inside the large neighbourhood balls of real images in sparse regions of feature space, while recall, which uses the damaged set's own balls, drops.

Copies of training images scored like real test digits on every feature-based score (IS 9.479, FID 3.161, recall 0.867). A generator that memorised its training set would look perfect to all of them. Only the pixel distance to the nearest training image saw it: 0.00, against 4.49 for real test digits.



Top: FID against sample size, in random and in stored order; IS, FID and recall of the damaged sets. Bottom: the ten repeated images; FID and recall of three GANs during training, against real test images; samples at epoch 20.

WHY CHOOSE THIS?

Used together, the scores cover each other: FID and recall see missing variety, the nearest-image test sees copying.

WHY NOT THIS? (TRADEOFFS)

Used alone, each has a blind spot: IS rewards collapse within a class, precision can reward damage, none of the feature scores sees memorisation.

5. Scoring real GANs during training

DEFINITION — start here, no prior knowledge assumed

Three GANs from earlier in the week are retrained and scored at seven epochs: Day 134's dense GAN with Adam's b1 at 0.9 and at 0.5, and Day 136's DCGAN with b1 0.5. All use 20,000 training images, batches of 128, learning rate 0.0002, seed 0.

REAL-WORLD SCENARIO

Weighing a growing child every few months instead of once at the end: the path shows things the last number cannot.

IMPLEMENTATION

```
def train_scored(G, D, X, sc, b1, epochs, seed, at, k=64, batch=128):
    oG, oD = d134.AdamGAN(G.net, 2e-4, b1), d134.AdamGAN(D.net, 2e-4, b1)
    rng = np.random.RandomState(seed); Zfix = np.random.RandomState(9).standard_normal((2000,
        k)).astype(np.float32)
    log, t0 = [], time.perf_counter()
    for ep in range(1, epochs + 1):
        o = rng.permutation(len(X))
        for s in range(0, len(X) - batch + 1, batch):
            d136.net_gan_step(G, D, oG, oD, X[o[s:s + batch]], rng, k)
        if ep in at:
            F = d136.net_generate(G, Zfix); log.append((ep, sc(F), F[:16].copy(),
                time.perf_counter() - t0))
    return log
```

ACTUAL OUTPUT

```
each trained as before: 20,000 training images (0-19,999), batches of 128, learning rate 0.0002, seed 0;
    scored on
2,000 images from fixed noise. Note: training images 10,000-19,999 are both in the GANs' training set and in
    the
nearest-image pool, so 'nearest' measures closeness to images each GAN saw
dense, b1 0.9:  epoch      IS      FID  precision  recall  nearest
                    1   1.053  364.008      0.993   0.000     8.20
                    2   1.034  397.757      1.000   0.000     8.55
                    3   1.024  402.780      1.000   0.000     9.19
                    5   1.076  433.346      1.000   0.000     9.02
                   10   1.007  411.688      1.000   0.000     9.07
                   15   1.028  422.625      0.996   0.000     9.19
                   20   1.208  391.481      1.000   0.000     6.40
                (68 s)
dense, b1 0.5:  epoch      IS      FID  precision  recall  nearest
                    1   1.114  348.662      0.997   0.000     7.18
                    2   1.099  349.097      1.000   0.000     7.03
                    3   1.189  427.703      1.000   0.000     4.75
                    5   1.239  422.536      1.000   0.000     4.02
                   10   3.919  114.752      0.849   0.004     5.69
                   15   4.768   66.322      0.680   0.203     5.75
                   20   4.918   53.291      0.582   0.355     5.75
                (63 s)
DCGAN, b1 0.5:  epoch      IS      FID  precision  recall  nearest
                    1   1.105  505.076      0.998   0.000     6.61
                    2   1.233  445.236      1.000   0.000     4.49
                    3   1.309  403.381      1.000   0.000     5.71
                    5   3.718  135.570      0.639   0.051     5.93
                   10   4.790   77.788      0.235   0.355     6.45
                   15   5.046   54.687      0.353   0.460     6.40
                   20   5.004   51.747      0.383   0.518     6.53
                (2493 s)
Actual output from this lesson's run.
```

What it means

With b1 0.9 the dense GAN never left collapse: IS about 1 throughout (1.208 at epoch 20), recall 0.000 at every epoch, and precision 0.993 to 1.000. Precision rewarded a generator producing one realistic-looking kind of image. Every GAN went through the same phase at first: precision near 1 and recall 0 for the first three epochs.

With b1 0.5 both the dense GAN and the DCGAN got out. At epoch 20 their FIDs are close (53.291 and 51.747), but they differ in kind: the DCGAN covers more of the real digits (recall 0.518 against 0.355) and fewer of its images land near real ones (precision 0.383 against 0.582). The DCGAN's recall rose earlier (0.355 at epoch 10, when the dense GAN's was 0.004). Both remain far from real test digits (FID 2.996, recall 0.884).

WHY CHOOSE THIS?

Two numbers, precision and recall, separate 'realistic' from 'varied', which FID mixes into one.

WHY NOT THIS? (TRADEOFFS)

Every score here depends on the feature network; a different judge could rank the GANs differently.

Interview preparation — technical questions

Q: How is FID computed?

A: Fit a Gaussian to a classifier's features of real and of generated images; $|\mu_1 - \mu_2|^2 + \mathrm{Tr}\,\Sigma_1 + \mathrm{Tr}\,\Sigma_2 - 2\,\mathrm{Tr}\,(\Sigma_1\Sigma_2)^{1/2}$.

Q: How do you take the matrix square root stably?

A: Eigen-decompose the symmetric Σ_1, then use $\Sigma_1^{1/2}\Sigma_2\Sigma_1^{1/2}$, which is symmetric and has the eigenvalues of $\Sigma_1\Sigma_2$.

Q: What are the Inception Score's limits?

A: From 1 to the number of classes; here ten images repeated 200 times scored 9.598, above real digits.

Q: Why must FIDs use the same sample size?

A: FID is biased upwards for small samples: real images scored 6.478 at 500 and 2.060 at 10,000.

Q: How are k-NN precision and recall defined?

A: Precision: share of fakes inside some real point's k-th-neighbour ball; recall: share of reals inside some fake's ball.

Interview preparation — theoretical questions

Q: Which score detects memorisation?

A: None of the feature scores here (copies: FID 3.161); a nearest-training-image distance did (0.00).

Q: Why did precision rise with noise?

A: Not tested here; a likely reason is that damaged images fall inside large real balls in sparse regions.

Q: What did the stored order of MNIST's test set do?

A: Its halves scored FID 12.006 and 6.146 on their own; prefixes mixed that with sample size.

Q: Which GAN was better at epoch 20?

A: By FID they were close (53.291, 51.747); by recall the DCGAN (0.518 against 0.355); by precision the dense GAN.

Q: Why are these 'FID-style' rather than FID?

A: The feature network is a small MNIST classifier, not Inception; the numbers are not comparable with published ones.

Key takeaway and what's next

KEY TAKEAWAY

Every score sees generated images through some judge, and each has a blind spot: the Inception Score rewards collapse within a class, precision can reward damage and collapse, FID depends on the sample size, and no feature-based score here noticed exact copies of training images. Use several, at a fixed sample size, on random samples, with a separate check for copying.

- Formulas checked: 1-D Frechet distance 10.000000; Inception Scores 10, 1, 1 and 5 on hand-made tables.
- Real test images, random order: FID 6.478 at 500 images, 2.996 at 2,000, 2.060 at 10,000.
- MNIST's test halves on their own: FID 12.006 and 6.146; a random 5,000: 2.250.
- Ten images repeated: IS 9.598 (real 9.407), FID 205.857, recall 0.000. Copies: FID 3.161, nearest 0.00.
- GANs at epoch 20: FID 53.291 (dense) and 51.747 (DCGAN); recall 0.355 and 0.518; dense with b1 0.9 never left collapse.

Suggested watch order, if starting from zero

- 1. What is Fréchet Inception Distance (FID)? (Data Science Made Easy) — a short explanation of FID's Gaussian fit and distance.
- 2. Frechet Inception Distance and Inception Score - AI Bits and Pieces (Shayan Fazeli) — FID and the Inception Score side by side, and what each measures.

Day 139 preview

- **The Week 20 build**: Day 137's conditional DCGAN from the exercise parts, scored with today's metrics against Day 133's Gaussian class models, and the forward diffusion process.